

ML-UNG: Metadata-Aware Multilevel Graph Navigation for Calibration-Free Filtered Approximate Nearest-Neighbor Search

Anonymous Author(s)

ABSTRACT

Filtered approximate nearest-neighbor search must reconcile vector proximity with metadata predicates. Unified Navigating Graph (UNG) [1] supports containment predicates by grouping vectors with identical label sets and connecting these groups through a label navigating graph. Because every navigation unit remains an exact-label group, however, UNG exposes only one granularity: as filter selectivity increases, a query must traverse many small groups and cross-group edges, and its graph work approaches a full-index search.

We present ML-UNG, which augments the exact-group graph with independently constructed metadata-aware overlays. At level ℓ , a point-mass threshold T_ℓ partitions compatible label-Trie subtrees into coarser blocks; each block owns a local proximity graph, and blocks are connected by an LNG or a prefix-Trie topology. A query authorizes a block only when its labels are contained in the block root. Every search state carries an activation level and expands only edges owned by that level; if no useful upper block is authorized, ML-UNG invokes the unchanged level-zero search. The hierarchy therefore reduces redundant fine-group traversal without weakening predicate eligibility or mixing edges across levels.

To eliminate manual choices of layer count and block size, Degree-Ratio Hierarchy derives thresholds, depth, topology, and a next-scale routing gate from dataset cardinality and existing graph-degree budgets. It requires no query distribution, learned model, latency measurement, or Recall calibration. We further batch irregular intra-block and cross-block construction on the GPU. At equal measured Recall@10, the automatically derived two-level plan improves QPS by 14.40–26.70 $\times$ over the single-scale baseline on Amazon at 60–99% selectivity, while the fastest GPU-assisted base-plus-hierarchy build is 1.35 $\times$ faster than the original CPU base builder. Held-out results also expose a dataset-dependent routing failure mode; we therefore claim substantial gains in the target regime rather than universal dominance.

ACM Reference Format:

Anonymous Author(s). 2026. ML-UNG: Metadata-Aware Multilevel Graph Navigation for Calibration-Free Filtered Approximate Nearest-Neighbor Search. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Filtered vector search returns the nearest vectors satisfying a predicate over metadata. For a containment query, the query supplies a set of labels and a vector is eligible only if its label set contains every query label. Graph indexes for this problem must provide both geometric navigation and predicate reachability. The two signals do not coincide: nearby vectors may violate the predicate, while eligible vectors may be scattered across many label sets.

UNG [1] resolves this mismatch by introducing an explicit label navigation plane. It groups vectors that have identical label sets, builds a proximity graph inside each group, connects minimal strict-superset groups with an LNG, and realizes those group edges as vector edges. For a containment query, a label Trie helps locate minimal observed supersets, after which one bounded graph search follows only predicate-compatible directions. This is the right starting point for our work: we retain UNG's exact groups, its containment direction, and its unified vector-search interface.

The limitation we address is different from predicate correctness. UNG has one label-navigation granularity: every vertex of its LNG denotes one exact label set. This fine partition is useful for narrow predicates, but a broad predicate may authorize hundreds of thousands of vectors spread over many exact groups. Search must still enter and traverse this fine graph. On our Amazon workload, the base index contains 482,387 observed groups for 602,453 vectors; at 95.02% selectivity the measured UNG-style base search visits 451,225 vertices and scans 3.97 million edges per query (Table 13). Merely replacing the LNG by a sparser graph does not solve the problem: it can remove the containment paths needed for Recall.

Our key observation is that containment admits *safe coarse navigation*. Suppose a node of the canonical label Trie represents prefix R_b . Every exact-label group in its subtree contains R_b . Therefore, when $Q \subseteq R_b$, all direct members of a block rooted there satisfy the query. This gives a query-independent way to aggregate fine groups without weakening the filter. The resulting coarse graph is an overlay, not a replacement: the original exact-group graph remains available whenever no coarse block is useful.

ML-UNG turns this observation into a multiscale UNG. Increasing mass thresholds independently partition the label Trie into blocks. Each overlay connects its blocks using either the LNG cover relation or a prefix-Trie sparsifier. Exact authorization selects legal overlays, while candidates carry an activation level and expand only edges owned by that level. We deliberately separate this inter-block topology from entry-group discovery: UNG already uses a Trie during entry lookup, whereas our *Trie topology* means that Trie ancestry itself defines materialized inter-block edges. Table 1 summarizes the boundary between inherited and new mechanisms.

The hierarchy introduces a second problem: naively exposing both the number of layers and every block threshold replaces one

tuning knob with several. We therefore derive them with Degree-Ratio Hierarchy (DRH), a deterministic rule based only on dataset size and the graph's existing degree budgets. DRH never observes query frequencies, latency, Recall, or a selectivity histogram. A structural mass gate decides whether a legal overlay is large enough to use; rejected queries invoke the unchanged level-zero search.

This paper makes the following contributions.

- **Multiscale label navigation.** We extend UNG's single exact-group navigation plane with independently constructed, predicate-safe block overlays. The base UNG graph is preserved rather than repartitioned.
- **Auditable query semantics.** Exact block authorization, level-tagged states, level-local edge expansion, and structural fallback make it explicit when an overlay may participate and prevent hidden cross-level behavior. Topology and entry discovery remain separate experimental axes.
- **Calibration-free structure selection.** DRH derives layer count, block thresholds, per-layer topology, and the next-scale routing gate from N , R , and C , eliminating query-trace and performance calibration.
- **End-to-end evidence.** We evaluate measured equal-Recall crossings, expose both substantial high-selectivity gains and held-out regressions, and accelerate sidecar construction with batched GPU intra-block and cross-block primitives. We do not infer universal dominance from the favorable cases.

2 PROBLEM DEFINITION

Let $\mathcal{D} = \{(x_i, A_i)\}_{i=1}^N$, where $x_i \in \mathbb{R}^d$ and $A_i \subseteq \mathcal{A}$ is a finite label set. For a query vector q , query labels Q , and result count K , containment filtering defines

$$\mathcal{E}(Q) = \{i \mid Q \subseteq A_i\}. \quad (1)$$

The exact answer is the K points in $\mathcal{E}(Q)$ with minimum distance to q . We report

$$\text{Recall@}K = \frac{|R_K(q, Q) \cap R_K^*(q, Q)|}{K}, \quad (2)$$

where R_K is the approximate result and R_K^* is exact filtered ground truth.

The empty label set is a valid predicate rather than a missing value: $\mathcal{E}(\emptyset) = \mathcal{D}$. Trie entry discovery therefore returns the minimal terminal frontier reachable from the root. We precompute this frontier when the Trie is built or loaded, so an empty predicate costs $O(|F_{\text{root}}|)$ rather than a full Trie scan. Returning no entries for this case would change containment semantics and can silently corrupt broad selectivity workloads.

Vectors with the same label set form a group $G_S = \{i \mid A_i = S\}$. The group partition is exact and disjoint. A group topology is a directed graph over observed label sets. The base vector graph contains intra-group proximity edges and vector-level realizations of the group-topology edges.

3 FROM UNG TO MULTISCALE NAVIGATION

3.1 What UNG Provides

For containment search, UNG's label plane is the transitive reduction of the observed set-containment relation: its LNG contains an edge $S \rightarrow S'$ when $S \subset S'$ and no observed label set lies strictly between them [1]. Its data path has four steps: (1) partition vectors by exact label-set equality; (2) build a proximity graph within every group; (3) construct the LNG between groups; and (4) materialize vector-level cross-group edges along LNG edges. At query time, UNG uses a label Trie to find minimal observed supersets of Q , samples vector entry points from those groups, and searches the unified vector graph toward more specific label sets.

Two distinctions matter for the rest of this paper. First, the Trie in UNG is an index for label-set organization and entry discovery; the LNG, not Trie parenthood, defines cross-group connectivity. Second, all LNG vertices are exact-label groups, so UNG provides one navigation scale. Our level-zero LNG configuration preserves that organization and the same minimal-superset entry definition. We focus on containment predicates; extending the overlays to the other predicate families supported by UNG requires separate authorization rules and is outside our claims.

3.2 Design Requirements

Adding a hierarchy is useful only if it preserves three boundaries. The hierarchy determines which vectors may share an overlay; topology determines which overlay blocks are adjacent; entry discovery determines only where search starts. Treating a Trie entry provider as a Trie graph, or changing block size while claiming to isolate layer count, confounds these effects. Our method schema therefore records

$$M = (H, \tau_0, \tau_1, \dots, \tau_L, e, r), \quad (3)$$

where H is the threshold sequence, τ_0 is the base topology, τ_ℓ is layer ℓ 's topology, e is the entry strategy, and r is the routing policy.

Orthogonality here is an interface and experimental property, not a claim that the factors have no interaction. An LNG entry routine returns minimal observed supersets under set containment, relying on LNG edges to reach other eligible supersets. A prefix-Trie topology omits non-prefix containment edges; its native entry routine must instead seed the minimal compatible terminal on each surviving prefix branch. Crossed configurations remain legal and are valuable controls, but an entry frontier designed for one reachability relation can lose Recall on the other. We therefore report Recall before comparing their QPS rather than treating entry discovery as a constant-time plug-in.

4 MULTILEVEL INDEX

4.1 Method Overview

ML-UNG consists of the unchanged level-zero exact-group graph plus zero or more independently serialized overlays. Building one overlay has three steps: partition the canonical label Trie by point mass, select one inter-block topology, and materialize intra-block and cross-block vector edges. Processing a query also has three

Table 1: Method boundary between UNG and ML-UNG. “Trie topology” in this paper denotes materialized inter-block edges and must not be confused with UNG’s Trie-assisted entry lookup.

Component	UNG [1]	ML-UNG (this work)
Navigation unit	One group per exact label set	Exact groups at level zero plus independently partitioned coarse blocks
Group/block connectivity	One LNG over exact groups	Per-layer LNG or prefix-Trie topology; every edge has one owner level
Entry discovery	Trie-assisted minimal-superset groups	Original/optimized LNG-semantic provider or topology-aware Trie frontier
Query routing	Start from matching entry groups and search one unified graph	Exact overlay authorization, optional structural mass gate, and base fallback
Search state	Vector identifier in one graph	(vector, distance, level); expansion is level local
Hierarchy parameters	Not applicable	DRH derives depth, thresholds, topology, and routing gate without query calibration
Construction	Per-group graph plus LNG-guided cross-group edges	Preserved base build plus independently serialized hierarchy sidecar and GPU batching

steps: obtain the normal base entry groups, authorize blocks by exact containment, and run one bounded queue whose states remember their owning level. If routing rejects all overlays, ML-UNG calls the level-zero routine before creating any overlay seed. If an overlay is active, its states never descend into base edges or ascend into another overlay. The hierarchy therefore changes navigation granularity without changing predicate semantics or silently changing the base graph.

This separation is central to our evaluation. A zero-layer Trie experiment changes actual group connectivity. A Trie entry-provider experiment changes only the initial frontier. A one- versus two-layer experiment changes the number of available granularities while holding the finer partition fixed. These are different interventions and are not labeled interchangeably.

4.2 Predicate-Safe Coarsening by Trie Mass

We build a canonical prefix trie over sorted group label sets. Each terminal stores one exact group and its point mass. For a threshold T_ℓ , a bottom-up pass maintains uncovered mass. When a non-root trie node has uncovered mass greater than T_ℓ , it becomes a block root and its uncovered mass is reset before propagating upward. The block owns terminal groups in its subtree until another root at the same layer is encountered. Each layer runs this partition independently; adding a coarse layer therefore does not change the blocks or identifiers of any finer layer.

For example, exact groups $\{a, b\}$, $\{a, c\}$, and $\{a, b, d\}$ can be direct members of a block rooted at prefix $\{a\}$. Query $Q = \{a\}$ authorizes this block and may navigate the three groups at the coarser scale. Query $Q = \{a, b\}$ does not authorize the $\{a\}$ block, because $Q \not\subseteq \{a\}$; it therefore uses a more specific authorized block or the base graph. The same containment test provides both predicate safety and query-dependent scale selection without estimating selectivity.

The build stores direct point count, full source-subtree count, common labels, an entry point, and a nearest containing parent in the next coarser layer. The parent relation is metadata for inspection; query traversal does not use it to promote candidates across layers.

Algorithm 1 Independent block partition at threshold T

Require: Canonical label trie $\mathcal{T}$ with terminal masses $w(v)$

Ensure: Disjoint direct-member blocks at one layer

- 1: $u(v) \leftarrow w(v)$ for every trie node v
- 2: **for** nodes v in reverse topological order **do**
- 3: $u(v) \leftarrow w(v) + \sum_{c \in \text{child}(v)} u(c)$
- 4: **if** v is not the root and $u(v) > T$ **then**
- 5: emit a block rooted at v
- 6: $u(v) \leftarrow 0$
- 7: **for** each emitted root b **do**
- 8: assign terminals below b until another emitted root is reached

4.3 Layer-Local Topologies

For each materialized layer, we support two group-to-group topologies. **LNG** connects every block root to its minimal strict label supersets within that layer. **Trie** connects a terminal block to the nearest terminal descendants along canonical prefix-trie branches. Both point from a less specific label set to a more specific one, but Trie is a strict sparsifier and need not preserve every LNG reachability path. We therefore treat its quality as an empirical property, not a semantic equivalence.

Each block owns a local vector graph and vector-level inter-block edges. Edges are serialized with their owner block and activation level. The base topology is built independently and remains available for exact fallback.

Algorithm 2 makes the Trie topology precise. A preorder scan carries the nearest terminal ancestor. Consequently, every non-root terminal has at most one incoming Trie edge and a terminal may have several outgoing edges, one to the first terminal on each descendant branch. The result contains $O(G)$ edges, but unlike LNG it does not connect set-containment relations whose sorted label sequences are not prefix related. For a block layer, terminals in the algorithm are the block roots emitted by Algorithm 1; traversal stops at the first nested block root, so no edge crosses a layer boundary.

Algorithm 2 Prefix-Trie group topology**Require:** Canonical label trie $\mathcal{T}$ and terminals $g(v)$ **Ensure:** Directed topology E_{Trie}

```

1:  $E_{\text{Trie}} \leftarrow \emptyset$ 
2: push (root( $\mathcal{T}$ ),  $\perp$ ) onto a stack
3: while the stack is nonempty do
4:    $(v, a) \leftarrow \text{Pop}(\text{stack})$ 
5:   if  $v$  is a terminal then
6:     if  $a \neq \perp$  then
7:        $E_{\text{Trie}} \leftarrow E_{\text{Trie}} \cup \{(g(a), g(v))\}$ 
8:      $a \leftarrow v$ 
9:   for  $c \in \text{child}(v)$  do
10:    push  $(c, a)$  onto the stack
11: return  $E_{\text{Trie}}$ 

```

4.4 Entry-Group Providers

The topology and the entry provider are separate interfaces. The *original LNG* provider sorts and filters candidate supersets to return the minimal observed supersets of Q . The *optimized LNG* provider computes the same definition with label-cardinality buckets and indexed containment checks. The *Trie* provider instead returns the first compatible terminal on every surviving prefix branch. It chooses the largest query label as a pivot, enumerates only its posting list, verifies the remaining labels by walking ancestors, and then descends each matched subtree until the first terminal on each branch. For $Q = \emptyset$, it returns the precomputed root terminal frontier. Thus all three providers return group identifiers consumed by the same vector-search interface, but the Trie frontier is deliberately topology aware and is not claimed to equal the LNG minimal-superset set.

4.5 Exact Authorization and Structural Eligibility

A query authorizes block b precisely when

$$Q \subseteq R_b, \quad (4)$$

where R_b is the block root's sorted label set. This test is independent of estimated selectivity and observed latency. Let h be the highest upper layer with at least one authorized block and let

$$\mathcal{A}_h(Q) = \{b \mid \text{level}(b) = h, Q \subseteq R_b\}. \quad (5)$$

Because direct members partition each layer, the authorized direct mass

$$M_h(Q) = \sum_{b \in \mathcal{A}_h(Q)} |P_b| \quad (6)$$

does not count a point once at a coarse layer and again at a finer layer.

We distinguish two query-free routing policies. DRH-v1 executes the multilevel backend whenever $\mathcal{A}_h(Q)$ is nonempty. DRH-v2 additionally requires $M_h(Q) \geq T_{L+1}$, where T_{L+1} is the next scale implied by the same recurrence that constructs the L materialized layers. If either test fails, the router invokes the unchanged base-only algorithm before any overlay seed is created. Otherwise the authorized layers are eligible to seed the multilevel search. Neither

Algorithm 3 Level-isolated multilevel query**Require:** Query (q, Q) , entry provider e , hierarchy H , budget L_s , optional mass threshold T_{gate}

```

1:  $B_Q \leftarrow \{b \in H : Q \subseteq R_b\}$ 
2: if  $B_Q$  has no upper block then
3:   return BASESEARCH( $q, Q, e, L_s$ )
4:  $h \leftarrow \max\{\text{level}(b) : b \in B_Q, \text{level}(b) > 0\}$ 
5:  $M_h \leftarrow \sum_{b \in B_Q: \text{level}(b)=h} |P_b|$ 
6: if  $M_h < T_{\text{gate}}$  then
7:   return BASESEARCH( $q, Q, e, L_s$ )
8: initialize a capacity- $L_s$  queue with block and entry-group seeds
9: while the queue has an unexpanded candidate  $(i, d, \ell)$  do
10:   pop the closest unexpanded candidate
11:   if  $\ell = 0$  then
12:      $E \leftarrow$  base edges of  $i$ 
13:   else
14:      $E \leftarrow$  edges of  $i$  owned by level  $\ell$ 
15:   for  $(i, j) \in E$  do
16:     if the edge owner is authorized and  $j$  is unseen at level  $\ell$  then
17:       insert  $(j, \text{dist}(q, x_j), \ell)$  into the bounded queue
18: filter queue results by  $Q$  and return the closest  $K$ 

```

policy reads query frequencies, latency, Recall, or a learned selectivity estimate.

4.6 Level-Isolated Bounded Search

Each candidate is a tuple (i, d, ℓ) containing a vector identifier, its distance, and an activation level. Level zero denotes the base graph; level $\ell > 0$ denotes one materialized overlay. Every authorized block contributes its stored entry point, and entry groups contribute vector entry points tagged with the level that owns them. All seeds share one queue of capacity L_{search} .

There is no edge fallthrough, implicit promotion, or mixed-level expansion. This provides a clean attribution of work to a layer. It does not imply that adding a layer is free: additional seeds can displace lower-layer candidates in the shared bounded queue.

4.7 Semantic Guarantees

The hierarchy does not make approximate graph search exact, but its routing and ownership rules provide three useful guarantees.

Authorization safety. If block b is authorized, then every direct-member point of b satisfies the query predicate. To see this, every member terminal is a descendant of the block root and therefore contains all labels in R_b . Authorization gives $Q \subseteq R_b$, so transitivity yields $Q \subseteq A_i$ for every direct member i of b . This claim is one-way: a block that is not authorized may still contain some eligible points, which remain reachable through another authorized layer or the base search.

Level-isolation invariant. Every expansion descended from a seed tagged ℓ scans only edges owned by ℓ . The invariant holds initially

because each seed is tagged with its owning level. During expansion, a special edge is accepted only when its owner level equals the current tag, and its successor retains that tag; a level-zero candidate reads only base edges. Induction over queue removals therefore excludes both implicit promotion and fallthrough. Separate visited sets per level ensure that the same vector identifier in two layers does not collapse the two search states.

Fallback equivalence. When the router rejects the overlay, the implementation invokes the same base-search routine with the same entry provider, entry groups, and L_{search} as the corresponding zero-layer method. Under deterministic execution, its returned neighbors and post-route search-work trace are therefore identical. Total latency can still increase by the authorization and mass tests themselves; we measure that cost rather than claiming that an unused hierarchy is free.

5 CALIBRATION-FREE HIERARCHY SELECTION

5.1 Degree-Ratio Hierarchy

DRH uses only values already required by index construction: N , maximum within-layer degree R , and cross-block edge budget C . We approximate the cost of a first scale T by

$$\Phi(T) = T + \frac{N}{T}, \quad (7)$$

whose continuous minimizer is $\sqrt{N}$. To stabilize discrete index layouts, we round to the nearest power of two:

$$T_1 = 2^{\lfloor \log_2 \sqrt{N} + \frac{1}{2} \rfloor}. \quad (8)$$

Successive thresholds use the existing degree ratio

$$\rho = \max\left(2, \text{round}\left(\frac{R}{C}\right)\right), \quad T_{\ell+1} = \rho T_\ell. \quad (9)$$

A scale is materialized while $N/T_\ell \geq C$. Ignoring only the implementation depth cap, the resulting number of layers is therefore

$$L = \max\left(0, 1 + \left\lfloor \log_\rho \frac{N}{CT_1} \right\rfloor\right). \quad (10)$$

Thus the depth is derived, not configured separately. The predeclared topology rule is LNG while the estimated number of blocks exceeds one full local neighborhood and Trie near the top:

$$\tau_\ell = \begin{cases} \text{LNG}, & N/T_\ell > R, \\ \text{Trie}, & \text{otherwise.} \end{cases} \quad (11)$$

For the authoritative Amazon setup, $N = 602,453$, $R = 64$, and $C = 4$, so DRH selects 1024:LNG, 16384:Trie before examining query performance. DRH-v2 sets its routing mass threshold to the next unmaterialized scale,

$$T_{\text{gate}} = T_{L+1} = \rho T_L. \quad (12)$$

This does not introduce an empirical knob: the value is already determined by N , R , and C . DRH-v1, retained as an ablation, is equivalent to $T_{\text{gate}} = 0$ with the nonempty-authorization requirement. We emphasize that Φ and the mass gate are query-independent

Algorithm 4 Degree-Ratio hierarchy with the DRH-v2 router

Require: N , local degree R , cross-block degree C

```

1:  $T \leftarrow$  nearest power of two to  $\sqrt{N}$ 
2:  $\rho \leftarrow \max(2, \text{round}(R/C))$ 
3: while  $N/T \geq C$  do
4:    $\tau \leftarrow$  LNG if  $N/T > R$ , else Trie
5:   append  $(T, \tau)$  to the hierarchy plan
6:    $T \leftarrow \rho T$ 
7:  $T_{\text{gate}} \leftarrow T$  ▷ the next unmaterialized scale
8: attach exact authorization and  $M_h \geq T_{\text{gate}}$  routing gate
9: return gated hierarchy plan
```

structural proxies, not proofs of runtime optimality; held-out oracle regret tests whether the complete policy transfers without query calibration.

Neither DRH variant is trained on query selectivity. The frozen held-out automatic-versus-manual comparison uses DRH-v1, because that candidate set was pre-registered before the DRH-v2 routing ablation; the same-binary DRH-v2 results are reported separately and are not retroactively substituted into the manual comparison. Evaluation must therefore separate the development dataset from held-out datasets and compare it with both the best per-workload method and the best single manual configuration used unchanged across workloads. Every held-out manual configuration receives the same exact authorization gate, so oracle regret measures only the cost of automatic depth, thresholds, and topology. Ungated DRH is retained solely as an ablation.

6 GPU-ASSISTED CONSTRUCTION

6.1 Irregular Construction Workload

Index construction contains two different irregular workloads. Intra-group construction operates on many groups with a long-tailed size distribution. Cross-group construction performs many $(n_q, n_x, d) \rightarrow \text{topK}$ tasks induced by topology edges. Launching a general GPU library once per group or group pair repeats packing, transfer, launch, and writeback overhead.

6.2 Base-Graph Pipeline

The base index is constructed before any hierarchy sidecar. Its pipeline has four semantic stages: build a vector graph inside every exact-label group, construct the group-to-group LNG, derive descendants and coverage, and materialize vector-level cross-group edges. Our optimization changes the implementation of these stages, not their ownership: group topology and metadata remain host structures, and the final vector adjacency is written in the CPU-compatible index format used by search.

The accelerated group-graph path batches irregular exact-label groups and uses the FastGrnd CUDA candidate generator, followed by relative-neighborhood-style pruning and configured degree repair. The cross-edge path converts host topology edges into descriptors for many (n_q, n_x, d) tasks. A conventional GPU base-line computes similarity tiles with SGEMM and performs top- k as a separate stage. Our fused path consumes the descriptors directly, combines distance evaluation with group-local top- k , and

Algorithm 5 Accelerated construction of the base unified graph

Require: Vectors X , exact-label groups $\{G_S\}$, degree R , cross degree C

Ensure: CPU-compatible unified graph A

- 1: construct group metadata and the declared group topology
- 2: **for** each memory-bounded batch of exact-label groups **do**
- 3: generate within-group candidates on the GPU
- 4: prune candidates and repair each group to the declared degree policy
- 5: append the resulting identifiers to host adjacency A
- 6: form descriptors $(G_S, G_{S'})$ for every group-topology edge
- 7: **for** each descriptor batch that fits the device budget **do**
- 8: evaluate distances and select group-local top- C identifiers jointly
- 9: compact identifiers and append cross-group edges to A
- 10: execute the declared additional-edge completion path
- 11: serialize, reload, and validate A and effective backend counters
- 12: **return** A

returns identifiers only because cross-edge distances are not stored. It keeps the base-vector array and norms resident when they fit the declared device budget; otherwise, bounded vector and query chunks are streamed. Thus the optimization removes repeated per-pair invocation and unnecessary score writeback without claiming a fully device-resident index builder: host adjacency materialization remains an explicit measured boundary.

Table 2 freezes the five base-build profiles used in the evaluation. In particular, `naive_gpu` and `paper_fused` retain CPU Vamana for intra-group graphs, isolating the cross-edge kernel change; `accelerated_gpu` additionally selects the CUDA group-graph implementation, optimized metadata, and the configured CPU exact-scan additional-edge path.

6.3 Hierarchy Intra-Block Graphs

For each hierarchy layer, the intra-block router sends small blocks to CPU exact top- k (or bounded complete) graphs, medium blocks to sampled CPU Vamana construction [4], and large blocks to a GPU adaptation of NN-Descent candidate generation [2] followed by relative-neighborhood-style pruning [8] and degree repair. The authoritative configurations use $n_{\text{small}} = 2048$, $n_{\text{large}} = 8192$, and 256 sampled candidates for the middle route. These fixed construction settings are shared by the compared hierarchy structures and are not inputs to DRH. A separate full-GPU profile disables this three-way router and sends every block above the small-block completion threshold to the CUDA path. Routing is determined by block cardinality and does not inspect downstream Recall. GPU output is compacted before device-to-host transfer, but the current system still materializes CPU-compatible adjacency lists. We report that conversion as an output-boundary cost rather than kernel work, and record any CUDA failure that falls back to CPU.

6.4 Grouped Fused Cross-Block Top- k

For every topology edge, source vectors need nearest target vectors. The GPU path groups descriptors across irregular tasks, keeps base vectors resident when capacity permits, addresses queries by

Algorithm 6 Hybrid GPU construction of a multilevel sidecar

Require: base vectors X , label trie $\mathcal{T}$, layer plan $\{(T_\ell, \tau_\ell)\}$, degree R , cross-edge budget C , device budget M

Ensure: validated CPU-compatible sidecar and backend evidence

- 1: **for** each layer ℓ independently **do**
- 2: $\mathcal{B}_\ell \leftarrow \text{PARTITIONTRIE}(\mathcal{T}, T_\ell)$
- 3: $\mathcal{E}_\ell \leftarrow \text{BUILDTOPOLGY}(\mathcal{B}_\ell, \tau_\ell)$
- 4: **for** each block $b \in \mathcal{B}_\ell$ **do**
- 5: **if** $|b| \leq n_{\text{small}}$ **then**
- 6: $A_b \leftarrow \text{EXACTTOPKCPU}(X_b, R)$
- 7: **else if** $|b| < n_{\text{large}}$ **then**
- 8: $A_b \leftarrow \text{SAMPLEDVAMANACPU}(X_b, R)$
- 9: **else**
- 10: $C_b \leftarrow \text{GPUCANDIDATEGRAPH}(X_b, M)$
- 11: $A_b \leftarrow \text{RNGPRUNEANDREPAIR}(C_b, R)$
- 12: $\mathcal{P} \leftarrow \{(u, v) : (u, v) \in \mathcal{E}_\ell\}$
- 13: **for** each memory-bounded descriptor batch $P \subseteq \mathcal{P}$ **do**
- 14: $A_P \leftarrow \text{FUSEDDISTANCE TOPK}(X, P, C, M)$
- 15: compact identifiers and append A_P to layer-local adjacency
- 16: record GPU work, explicit CPU fallbacks, phase times, and edge counts
- 17: serialize, reload, and structurally validate every declared layer
- 18: **return** sidecar and effective-backend evidence

global vector identifier, fuses distance and top- k selection, and writes identifiers only when distances are not consumed downstream. This design builds on GPU similarity-search work that identifies k -selection and memory traffic as central bottlenecks [5]. Large workloads use bounded streaming rather than assuming that the full vector array fits device memory. CPU fallback remains explicit and is recorded in build logs.

Algorithm 6 gives the end-to-end sidecar path. The routing decision depends only on block cardinality and declared memory limits. In particular, a GPU-labelled result is invalid when a requested GPU stage falls back silently; our validator checks the effective backend counters persisted by the builder.

Intra-block and inter-block execution are independently controlled. We therefore freeze the five hierarchy profiles in Table 3, rather than describing a sidecar with one undifferentiated GPU switch. The `full_gpu` profiles disable the three-way intra router and send every block above the bounded-complete threshold to CUDA. The WMMA variant changes the inter-block fused top- k to TF32 tensor-core execution; it does not redefine the hierarchy or the intra-block graph algorithm.

Layer-local and inter-block edges have independent backend choices, while all layers within a named profile use the same declared routing policy. A case is accepted only if its log proves the requested GPU work occurred, reports zero GPU intra-block fallback, proves GPU inter-block use when requested, and, for `full_gpu_wmma`, contains an observed `mode=tf32_wmma` execution. We then serialize and reload every declared layer and validate its structure. A predeclared quality cohort additionally re-queries the original and accelerated base indexes and the CPU, full-GPU, and full-GPU-WMMA sidecars for both studied hierarchy structures under the

Table 2: Frozen base-graph construction profiles. “Optimized metadata” denotes bucketed minimal-superset discovery, optimized LNG construction, epoch-based descendants, and direct descendant coverage.

Profile	Intra-group graph	Group metadata	Vector cross-group edges
original_cpu	CPU Vamana	Original sort/LNG, hash-BFS descendants, topological coverage	Original CPU builder
current_cpu	CPU Vamana	Optimized metadata	CPU Vamana cross-edge builder
naive_gpu	CPU Vamana	Optimized metadata	Batched GPU SGEMM plus separate top- k
paper_fused	CPU Vamana	Optimized metadata	Descriptor-grouped fused distance/top- k with identifier-only writeback
accelerated_gpu	FastGrndd CUDA path	Optimized metadata	Fused GPU cross edges plus CPU exact-scan additional edges

Table 3: Frozen hierarchy-sidecar construction profiles. CPU completion is retained below the declared small-block threshold in every profile.

Profile	Intra-block route	Inter-block route
cpu	Small exact/complete, medium sampled Vamana, large CPU Vamana	CPU
hybrid_gpu_intra	Small exact/complete, medium sampled Vamana, large FastGrndd-style CUDA	CPU
hybrid_gpu_intra_inter	Same hybrid intra route	Fused CUDA top- k
full_gpu	Bounded completion, then CUDA	Fused CUDA-core top- k
full_gpu_wmma	Bounded completion, then CUDA	Fused TF32-WMMA top- k

same formal equal-Recall protocol as the query study. Backend evidence is therefore necessary but not sufficient for a claimed construction result.

Because the hierarchy is an independent sidecar, its total build time satisfies

$$T_{\text{multilevel}} = T_{\text{base}} + T_{\text{sidecar}}. \quad (13)$$

We therefore compare total multilevel build against the original CPU build, and report sidecar overhead against the same accelerated base separately.

7 COMPLEXITY ANALYSIS

Let G be the number of exact groups, B_ℓ the number of blocks at layer ℓ , G_ℓ the number of exact-group memberships assigned at that layer, M_ℓ the number of direct member points represented at that layer, E_ℓ its block-topology edges, and D_ℓ the number of materialized vector edges. Building the label trie costs $O(\sum_g |S_g|)$ time and space. Each independent partition is linear in the trie plus emitted memberships, so all partitions cost

$$O\left(L|\mathcal{T}| + \sum_{\ell=1}^L G_\ell\right). \quad (14)$$

The stored hierarchy costs $O(\sum_\ell (B_\ell + G_\ell + D_\ell))$ excluding the base index. Trie topology has one incoming nearest-terminal edge per non-root terminal and hence $O(B_\ell)$ block edges, whereas explicit LNG construction can require pairwise containment checks and a denser transitive reduction in the worst case. We report the materialized edge counts rather than infer cost from these bounds.

For one query, exact block authorization by direct comparison is $O(\sum_b |R_b|)$; indexed Trie search can reduce the examined frontier, but we report measured authorization time rather than assuming a distribution. Bounded graph search stores $O(L_{\text{search}})$ candidates. Its dominant time is

$$O(D_q d + I_q \log L_{\text{search}}), \quad (15)$$

where D_q is the measured number of distance computations and I_q is the number of successful queue insertions. Since multiple authorized layers share the queue, neither quantity is guaranteed to decrease with depth.

8 EXPERIMENTAL METHODOLOGY

8.1 Questions

We ask: (Q1) When does multiscale navigation improve over the single-scale UNG structure at equal Recall? (Q2) Which effects come from topology, entry discovery, and hierarchy depth? (Q3) How close is DRH to manual tuning on development and held-out datasets? (Q4) Can GPU assistance keep total multilevel construction competitive with the original CPU pipeline? (Q5) Which phase and work counters explain each result?

8.2 Datasets and Workloads

The development dataset is Amazon with 602,453 vectors of dimension 768 and 482,387 observed label groups. Each workload contains 1,000 queries, uses $K = 10$, and has exact containment ground truth. The measured mean selectivities are 0.499249%, 0.903055%, 5.038117%, 9.906615%, 30.027242%, 60.047491%, 80.023699%, 95.019781%, and 99.000600%. Genome, Reviews, and VariousImg are held out for the automatic-policy study; their exact workload directories and fingerprints are recorded in the artifact manifest. The 99% Amazon workload contains 697 empty predicates and 303 singleton predicates $\{1\}$. We retain this mixture because both are legal containment queries, and validate the empty-predicate path separately before timing.

8.3 Methods and Fairness

The zero-layer principal comparison is LNG topology with optimized-LNG entry discovery versus Trie topology with Trie entry discovery. The factorial ablation independently varies zero/one/two layers, thresholds, LNG/Trie at each layer, and the three legal entry strategies: original LNG, optimized LNG, and Trie. Routed controls

Table 4: Held-out datasets and query-independent DRH plans. Selectivity characterizes the frozen queries and is not an input to DRH.

Dataset	N	d	Workload	Queries	Selectivity	DRH plan
Genome	108,077	512	query_minlen5_cov0.1k	3,000	3.365%	256:LNG, 4096:TRIE
Genome	108,077	512	query_minlen2_cov1k	3,000	6.292%	256:LNG, 4096:TRIE
Reviews	288,065	384	query_minlen5_cov0.1k	3,000	0.200%	512:LNG, 8192:TRIE
Reviews	288,065	384	query_minlen2_cov1k	2,864	4.115%	512:LNG, 8192:TRIE
VariousImg	758,935	512	query_minlen2_cov5k	3,000	10.252%	1024:LNG, 16384:TRIE

are outside the factorial because routing changes which search algorithm executes. For held-out automatic-policy evaluation, however, the exact gate is fixed for DRH and every manual-alternative hierarchy; the ungated DRH variant is reported separately.

We call the zero-layer LNG configuration *Plain* or *0L-LNG*. It preserves UNG’s exact-label groups, LNG cover relation, and unified vector graph, but it is not the unmodified upstream release: it uses our optimized, semantically equivalent minimal-superset provider and the common refactored execution path. This is the stronger control for the hierarchy because Plain and every overlay method share entry discovery and base-graph code. Reported multilevel speedups therefore isolate navigation scale; they are not presented as direct timings against the original UNG binary.

All query methods use the same data, query vectors, labels, exact ground truth, K , thread count, immutable search binary, graph backend, and Recall protocol. All primary query values use 100 query threads over the full frozen batch. The deadline evidence is explicitly screen-level: one cold run followed by two warm repeats, with QPS from warm-median batch time. Every repeat executes the same frozen query file in the same order; repeats are repeated measurements, not new random samples. We discard the cold repeat, derive primary QPS from the median warm batch time, and retain per-repeat values for dispersion estimates. Recall is not averaged to relax the quality target: every warm repeat at the selected operating point must meet the threshold. We select the smallest *measured* L_{search} for which every warm repeat reaches $\text{Recall}@10 \geq 0.90$; there is no interpolation or extrapolation. The screen pass locates crossings and a separate profile pass collects expensive counters. The current deadline artifact uses one discarded cold repeat and two warm repeats; a later formal pass is reserved for confidence intervals. If a method has no screen crossing, its crossing pass extends to the largest L_{search} measured for any method on that workload (or an explicitly declared common cap); only failure at that shared budget is reported as no crossing.

8.4 Metrics

Primary query metrics are warm-median batch latency and QPS. Bootstrap 95% confidence intervals are reserved for the later formal pass and are not claimed by the current deadline artifact. We additionally report entry-group search, entry-point setup, block authorization, graph search, residual time, visited nodes, scanned base/special edges, and distance computations. Build results use process wall time through validated files on disk. The full protocol

Table 5: Measured Amazon crossing and screen caps.

Selectivity	Plain crossing L	Screen cap L
0.499%	2,000	2,000
0.903%	2,200	2,200
5.038%	3,200	5,000
9.907%	26,000	64,000
30.027%	120,000	150,000
60.047%	260,000	320,000
80.024%	320,000	602,453
95.020%	320,000	602,453
99.001%	400,000	602,453

requires one cold build followed by at least five measured builds, interleaved CPU/GPU order, and peak host/device memory in a separate pass, and index bytes. Full-index cost is the sum of independently measured base and hierarchy stage medians; confidence intervals independently resample all three stages rather than pairing repeat identifiers across phases. GPU kernel timings are never substituted for construction wall time. GPU runs use an external exclusive lock when the host provides one. On the current host, where that tool is unavailable, each GPU case instead requires three consecutive idle device snapshots before timing; we retain those snapshots and label the result as an idle-preflight measurement rather than claiming lock-level exclusivity. The current deadline artifact reports completed base-stage timing and CPU sidecars separately; it makes no end-to-end GPU hierarchy claim before the remaining hierarchy repeats finish.

9 RESULTS

9.1 Amazon Query Performance

These are bounded screen-level results (one discarded cold repeat and two warm repeats), not formal confidence-interval estimates. At low selectivity the zero-layer Trie changes group connectivity and can be substantially faster; its loss at 10% and above shows that this is not a monotone hierarchy effect. The ungated one- and two-layer methods do not cross the target at 30%, while high-selectivity gains reach 27.99× and 27.57×. Exact upper routing preserves low-selectivity crossings but remains 0.64–0.99× of plain over 0.5–10%.

Table 6: Amazon QPS speedup at the smallest measured point whose two warm repeats both reach Recall@10 ≥ 0.90 . NC means no crossing in the measured budget.

Selectivity	Plain QPS	0-layer Trie	1-layer	2-layer ungated	2-layer gated
0.499%	2476.35	6.03×	NC	NC	0.86×
0.903%	4191.04	2.77×	NC	NC	0.99×
5.038%	544.93	18.43×	NC	NC	0.85×
9.907%	450.17	0.60×	NC	NC	0.64×
30.027%	52.77	NC	NC	NC	NC
60.047%	4.07	0.62×	13.89×	15.09×	14.40×
80.024%	1.80	0.33×	20.33×	27.57×	24.22×
95.020%	1.50	0.33×	27.99×	26.46×	26.70×
99.001%	0.79	0.56×	19.57×	21.34×	18.78×

Table 7: Fixed- $T_1 = 1024$ one-layer topology ablation with the same optimized-LNG entry provider. QPS is reported only at a measured Recall@10 ≥ 0.90 crossing; $R_{\max}$ exposes the quality reached when a crossing is absent.

Selectivity	LNG QPS	Trie QPS	Trie/LNG	LNG $R_{\max}$	Trie $R_{\max}$
0.499%	NC	NC	—	0.8878	0.8878
0.903%	NC	NC	—	0.8567	0.8464
5.038%	NC	NC	—	0.8928	0.8835
9.907%	NC	NC	—	0.8742	0.8477
30.027%	NC	NC	—	0.8359	0.8283
60.047%	56.52	56.70	1.003×	0.9511	0.9503
80.024%	36.65	37.08	1.012×	0.9346	0.9309
95.020%	42.06	36.82	0.875×	0.9289	0.9259
99.001%	15.40	14.34	0.931×	0.9325	0.9301

9.2 Automatic Versus Manual Hierarchies

The bounded held-out study compares DRH-v1 with five pre-registered manual alternatives. DRH-v2 was evaluated later as a separate same-binary routing ablation and is not substituted into this frozen candidate set. This is not a 35-case full oracle.

9.3 Structural Router Ablation

DRH-v2 is a same-binary ablation: it rejects an upper layer when direct mass at the highest authorized layer is below the next derived scale. The next-scale gate restores near-plain behavior on

Genome and the 0.2% Reviews workload, and improves VariousImg over DRH-v1, but it also rejects the useful overlay at 4.115% Reviews. Thus it reduces catastrophic overhead without establishing monotone dominance.

9.4 Mechanism Breakdown

The separate detailed-profile binary is explanatory only and does not contribute primary QPS. Edge counts are measured here; disabled light-stat counters are never interpreted as zero.

Table 8: Two-layer topology ablation at fixed $T_1 = 1024$, $T_2 = 16384$, optimized-LNG entry, and ungated routing. QPS appears only at a measured Recall@10 ≥ 0.90 crossing.

Sel.	LL QPS	LT QPS	TL QPS	TT QPS	LL $R_{\max}$	LT $R_{\max}$	TL $R_{\max}$	TT $R_{\max}$
0.499%	NC	NC	NC	NC	0.8878	0.8878	0.8878	0.8878
0.903%	NC	NC	NC	NC	0.8567	0.8567	0.8464	0.8464
5.038%	NC	NC	NC	NC	0.8929	0.8928	0.8836	0.8836
9.907%	NC	NC	NC	NC	0.8484	0.8492	0.8220	0.8229
30.027%	NC	NC	NC	NC	0.8385	0.8393	0.8333	0.8324
60.047%	56.37	61.40	57.00	56.42	0.9529	0.9538	0.9523	0.9525
80.024%	35.44	49.70	35.88	35.94	0.9357	0.9362	0.9341	0.9313
95.020%	39.61	39.75	36.99	36.68	0.9306	0.9310	0.9294	0.9271
99.001%	14.69	16.79	14.41	14.28	0.9338	0.9346	0.9325	0.9313

Table 9: Screen-level gated DRH-v1 results at Recall@10 ≥ 0.90 .

Dataset	Selectivity	Plain QPS	DRH-v1/plain	DRH-v1/best manual	Best manual plan
Genome	3.365%	153.65	0.954×	1.008	256,4096:trie,trie
Genome	6.292%	85.45	0.982×	1.007	256:lng
Reviews	0.200%	19319.56	1.014×	0.997	1024,16384:lng,trie
Reviews	4.115%	354.78	1.004×	0.980	1024,16384:lng,trie
VariousImg	10.252%	2657.04	0.205×	0.218	1024:lng

Table 10: One fixed DRH-v1 plan versus one fixed manual plan per held-out dataset (geometric-mean QPS).

Dataset	Workloads	DRH-v1 QPS	DRH-v1/manual
Genome	2	110.86	1.013
Reviews	2	2642.45	0.989
VariousImg	1	543.74	0.218

9.5 Amazon Stage and Work Breakdown

This profile-only pass reuses each method’s measured performance operating point. Rows marked max use the best measured point because Recall@10 did not cross 0.90; they explain work but are not equal-Recall speed comparisons. NC denotes a declared profile timeout at the fixed 3,300-second per-case cap; no partial counters are used.

9.6 Construction

The completed held-out sidecars below are CPU query-enablement builds, not evidence of GPU hierarchy speedup. For context, the completed base-index timing stage measured 190.61s for original CPU and 53.18s for accelerated_gpu, a 3.58× base-stage speedup. The campaign manifest contains 41 completed cases; the repeated hierarchy and composed results follow.

10 DISCUSSION

10.1 Why More Layers Are Not Monotone

Level isolation prevents an upper candidate from scanning lower-level edges, but does not make an extra layer cost-free. Authorized block seeds enter the same bounded queue as base or finer-layer seeds. They consume distance computations and can displace candidates before useful regions are expanded. DRH-v1 removes this overhead only for queries that authorize no upper block. DRH-v2 also rejects an authorized overlay whose highest-layer direct mass is below the next hierarchy scale. This remains a structural rule rather than a latency predictor, so it cannot guarantee that every accepted overlay is profitable.

10.2 Trie Topology Is a Sparsifier, Not an LNG Alias

Canonical prefix adjacency is cheaper and can be highly effective when query entries align with its branches. It omits non-prefix subset relations, however, so large L_{search} cannot recover vectors in unreachable graph regions. This also explains why crossing an LNG-style minimal-superset entry frontier with a Trie topology can reduce work yet lower Recall: fewer seeds do not cover every prefix branch that the Trie graph keeps disconnected. We therefore report no-crossing outcomes explicitly and pair Recall with entry count, visited-node, scanned-edge, and distance-computation counts.

Table 11: Available same-binary DRH-v2 screen results.

Dataset	Selectivity	DRH-v1 QPS	DRH-v2 QPS	v2/plain
Genome	3.365%	144.50	142.57	0.992×
Genome	6.292%	83.73	83.16	1.005×
Reviews	0.200%	17445.16	18335.39	1.007×
Reviews	4.115%	368.07	351.49	0.972×
VariousImg	10.252%	547.01	884.18	0.336×

Table 12: Detailed profile at each held-out dataset's broader workload (ms/query and mean work per query).

Dataset	Method	Graph ms	Visited	Edges	Distances	Active
Genome	plain	0.124	123.8	154.3	4538.9	0.0%
Genome	DRH-v1	0.199	185.1	591.8	4601.4	1.8%
Reviews	plain	6.756	4415.8	15126.2	8239.3	0.0%
Reviews	DRH-v1	6.504	3500.1	25605.4	7312.0	18.2%
VariousImg	plain	35.098	18656.0	119045.5	18673.1	0.0%
VariousImg	DRH-v1	172.065	27356.1	251144.0	27393.7	37.4%

10.3 Limitations

DRH uses a symmetric static cost proxy, existing degree budgets, exact authorization, and a next-scale mass gate; it does not use a learned cost model or query frequencies. The held-out study is required to distinguish genuine structural generalization from an Amazon-specific coincidence. The GPU pipeline retains a host adjacency materialization boundary, and full device-resident construction is future work. All four evaluated datasets instantiate two DRH layers, so the held-out study tests transfer of thresholds and topology but does not by itself establish performance when the derived depth changes. Finally, the current evaluation covers containment predicates; other predicate semantics require a new authorization proof.

11 RELATED WORK

11.1 Filtered Graph Search and UNG

Graph-based ANN systems such as HNSW [6] and DiskANN [4] use navigable proximity graphs to obtain high-recall search with bounded expansion. Filtered-DiskANN [3] and ACORN [7] integrate metadata constraints through filter-aware construction or predicate-agnostic graph expansion.

UNG [1] is our closest baseline. Its contribution is an LNG whose vertices are observed exact label sets and whose edges encode minimal strict-superset relations; group-level relations are then realized in one unified vector graph. ML-UNG does not replace this containment abstraction. It retains it at level zero and adds independently partitioned block overlays above it. Authorization follows from the common labels at a block root, and a level tag restricts each expansion to one overlay. This is also distinct from HNSW's

hierarchy: HNSW samples vector vertices to accelerate geometric navigation, whereas our layers aggregate label-space regions and are selected by an exact predicate test. Prefix-Trie connectivity is an optional inter-block sparsifier in our framework, not a relabeling of UNG's existing Trie-assisted entry lookup.

11.2 Graph Construction

NN-Descent [2] provides an efficient iterative basis for approximate k -NN graph construction, while relative neighborhood graphs [8] motivate diversity-oriented proximity pruning. GPU similarity search supplies high-throughput distance and top- k primitives [5], but per-group invocation can be dominated by packing and launch overhead. Our grouped execution strategy amortizes these costs across irregular tasks while preserving the filtered graph's output semantics and measuring quality end to end.

12 CONCLUSION

UNG established exact-label groups and containment-directed cross-group edges as a unified substrate for filtered graph search. ML-UNG addresses the next bottleneck: one exact-group navigation scale cannot efficiently serve every eligible mass. Predicate-safe Trie blocks provide coarse overlays while the original group graph remains level zero. Exact authorization and level-local edges make this multiscale behavior auditable, and DRH derives depth, thresholds, topology, and routing without query calibration. The accompanying protocol evaluates measured Recall crossings and separates kernel, component wall time, and stage-composed full-index construction costs. The current evidence supports multilevel acceleration for broad Amazon predicates, but not a universal dominance claim: DRH-v1 is near plain and near the frozen manual set on Genome

Table 13: Amazon detailed profile. Times are ms/query; work counters are means/query.

Sel.	Op.	Method	ELS	Entry	Auth.	Graph	Visited	Edges	Distances
0.499%	cross	0L-LNG	20.352	0.195	0.000	2.076	1966.8	23458.9	2150.2
0.499%	cross	0L-Trie	0.645	1.769	0.000	0.868	988.6	11047.0	2764.4
0.499%	max	1L-LNG	19.665	0.216	0.015	4.418	1860.2	23296.2	2043.9
0.499%	max	1L-Trie	16.188	0.224	0.012	4.578	1859.2	23263.7	2042.9
0.499%	cross	2L-gated	20.455	0.194	0.003	2.109	1968.1	23464.7	2151.5
9.907%	cross	0L-LNG	36.631	0.646	0.000	98.747	25024.2	189382.5	25402.9
9.907%	cross	0L-Trie	3.778	14.744	0.000	323.081	38926.2	351908.0	44682.9
9.907%	max	1L-LNG	33.451	0.645	0.017	644.477	34782.5	821915.0	35184.9
9.907%	max	1L-Trie	32.969	0.576	0.015	587.339	32344.1	757831.0	32746.5
9.907%	cross	2L-gated	31.648	0.685	0.023	301.608	28532.4	449218.0	28936.2
30.027%	cross	0L-LNG	251.380	3.963	0.000	1376.910	91953.6	580600.5	93654.0
30.027%	max	0L-Trie	8.381	30.629	0.000	2420.260	88466.5	497828.0	98769.0
30.027%	max	1L-LNG	220.485	1.887	0.017	1765.145	61473.4	1112780.0	63220.3
30.027%	max	1L-Trie	226.654	1.963	0.015	1662.880	58664.4	1021480.0	60411.4
30.027%	max	2L-gated	234.946	2.091	0.013	1755.115	64374.7	978653.5	66124.7
60.047%	cross	0L-LNG	2123.720	33.187	0.000	21357.550	254657.0	1988540.0	265484.5
60.047%	cross	0L-Trie	10.670	63.353	0.000	39255.750	261406.5	1453445.0	278174.0
60.047%	cross	1L-LNG	1415.925	8.882	0.017	200.452	45762.3	218706.0	56600.6
60.047%	cross	1L-Trie	1465.900	8.881	0.013	192.047	44045.4	200138.0	54883.6
60.047%	cross	2L-gated	1408.655	9.064	0.017	195.087	45607.8	215753.5	56454.1
95.020%	cross	0L-LNG	4918.875	61.074	0.000	89240.100	451225.0	3967260.0	467715.0
95.020%	NC	0L-Trie	–	–	–	–	–	–	–
95.020%	cross	1L-LNG	2307.095	12.576	0.018	113.150	44157.5	170659.0	60663.6
95.020%	cross	1L-Trie	2313.385	13.444	0.014	299.972	67643.4	309628.0	84149.5
95.020%	cross	2L-gated	2317.090	12.571	0.024	113.679	43995.0	170239.0	60513.8

Table 14: Automatic DRH CPU sidecar construction.

Dataset	Wall s	Blocks	Upper	Edges
Genome	44.2	264	18	10,088,461
Reviews	120.4	217	16	27,590,479
VariousImg	633.2	190	10	80,034,774

and Reviews, and substantially worse on VariousImg; the separate DRH-v2 router limits some regressions but remains non-monotone. Repeated construction measurements show that the fastest composed base-plus-hierarchy path is 1.35× the original CPU base builder (bootstrap 95% CI [1.33, 1.36]); stage medians are measured independently.

Evidence provenance. Full paths and SHA256 digests for 22 validated inputs are stored in the deadline artifact manifest; their ordered aggregate digest begins 076a8fba9fd72f43.

REFERENCES

- [1] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–27.
- [2] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k -Nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web*. Association for Computing Machinery, New York, NY, USA, 577–586. <https://doi.org/10.1145/1963405.1963487>
- [3] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, 3406–3416. <https://doi.org/10.1145/3543507.3583552>
- [4] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 11 pages.
- [5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [6] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [7] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27. <https://doi.org/10.1145/3654923>
- [8] Godfried T. Toussaint. 1980. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12, 4 (1980), 261–268. [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7)

Table 15: Repeated Amazon hierarchy-sidecar construction measurements. Resource values come from a separate profiling run and do not enter timing medians.

Backend	Repeats	Wall median (s)	CV	Speedup/CPU	Peak RSS (MiB)	Peak GPU (MiB)
cpu	2	1000.64	0.005	1.00×	5001.5	0.0
full_gpu	2	105.90	0.007	9.45×	4178.1	2286.0
full_gpu_wmma	2	1011.13	0.000	0.99×	4223.5	2362.0
hybrid_gpu_intra	2	90.49	0.008	11.06×	4880.7	1598.0
hybrid_gpu_intra_inter	2	132.94	0.006	7.53×	4255.6	2286.0

Table 16: Composed end-to-end construction: accelerated base stage plus hierarchy sidecar versus the original CPU base builder. Stage medians are measured independently.

Hierarchy backend	Stage repeats	Original CPU (s)	Composed (s)	Speedup	95% CI
cpu	2	190.38	1051.42	0.18×	[0.18, 0.18]
full_gpu	2	190.38	156.69	1.21×	[1.20, 1.23]
full_gpu_wmma	2	190.38	1061.91	0.18×	[0.18, 0.18]
hybrid_gpu_intra	2	190.38	141.27	1.35×	[1.33, 1.36]
hybrid_gpu_intra_inter	2	190.38	183.72	1.04×	[1.03, 1.05]

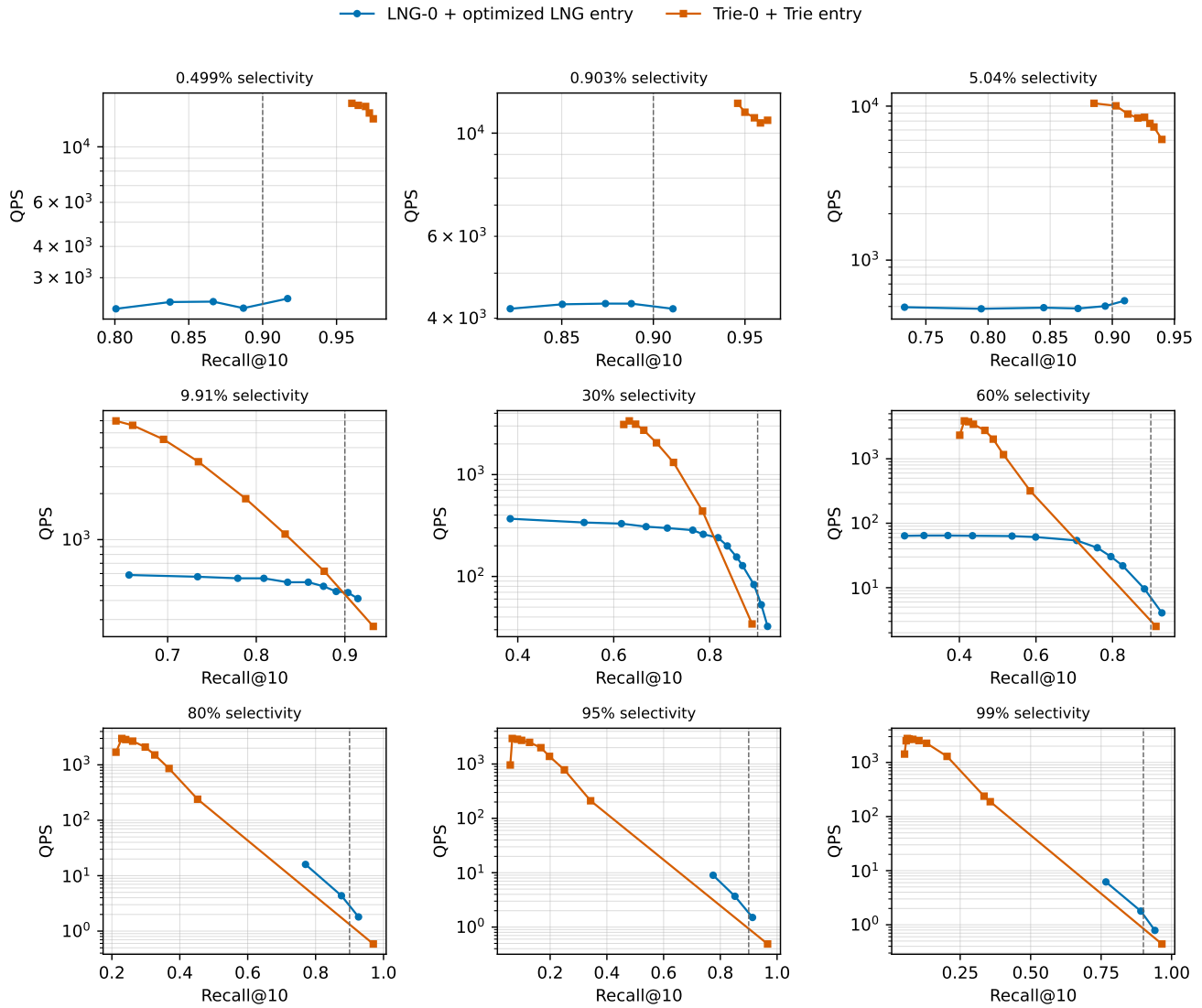


Figure 1: Measured Recall@10-QPS curves for the zero-layer LNG/Trie comparison. Markers are executed points; curves are not interpolated.

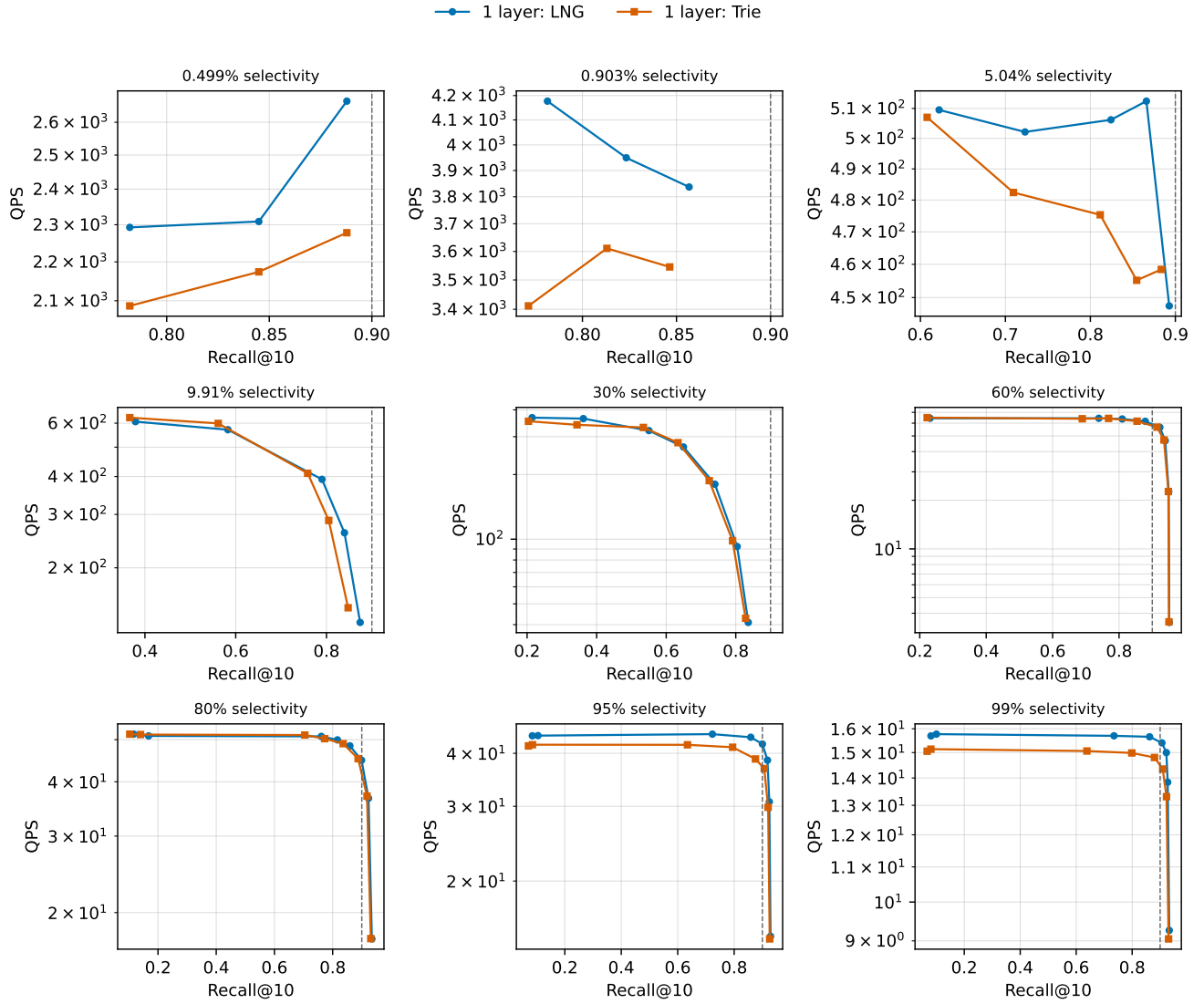


Figure 2: Measured Recall@10-QPS curves for one-layer LNG/Trie topology at fixed threshold and entry provider. Markers are executed points; curves are not interpolated.

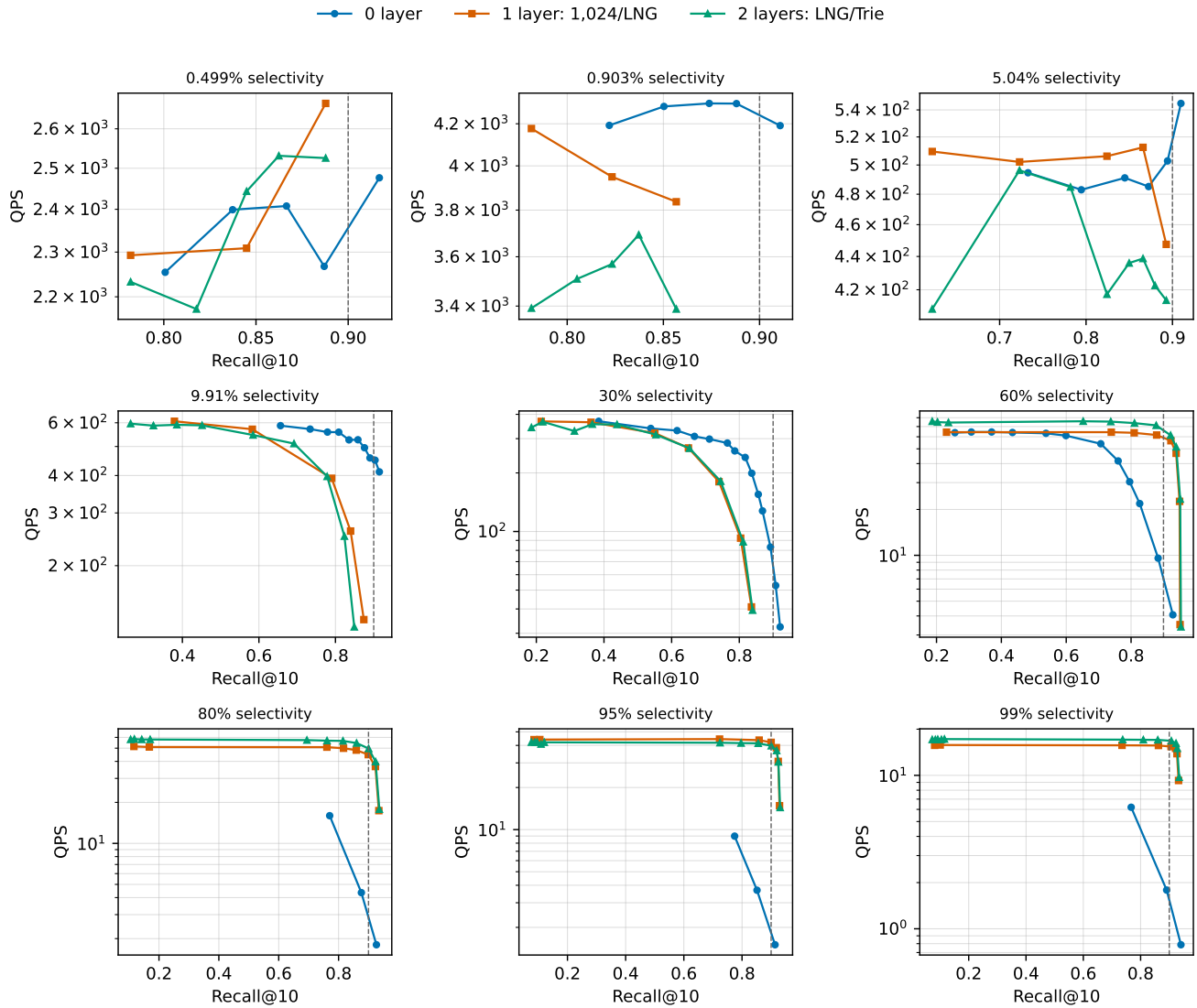


Figure 3: Measured Recall@10-QPS curves for the representative zero/one/two-layer comparison. Markers are executed points; curves are not interpolated.

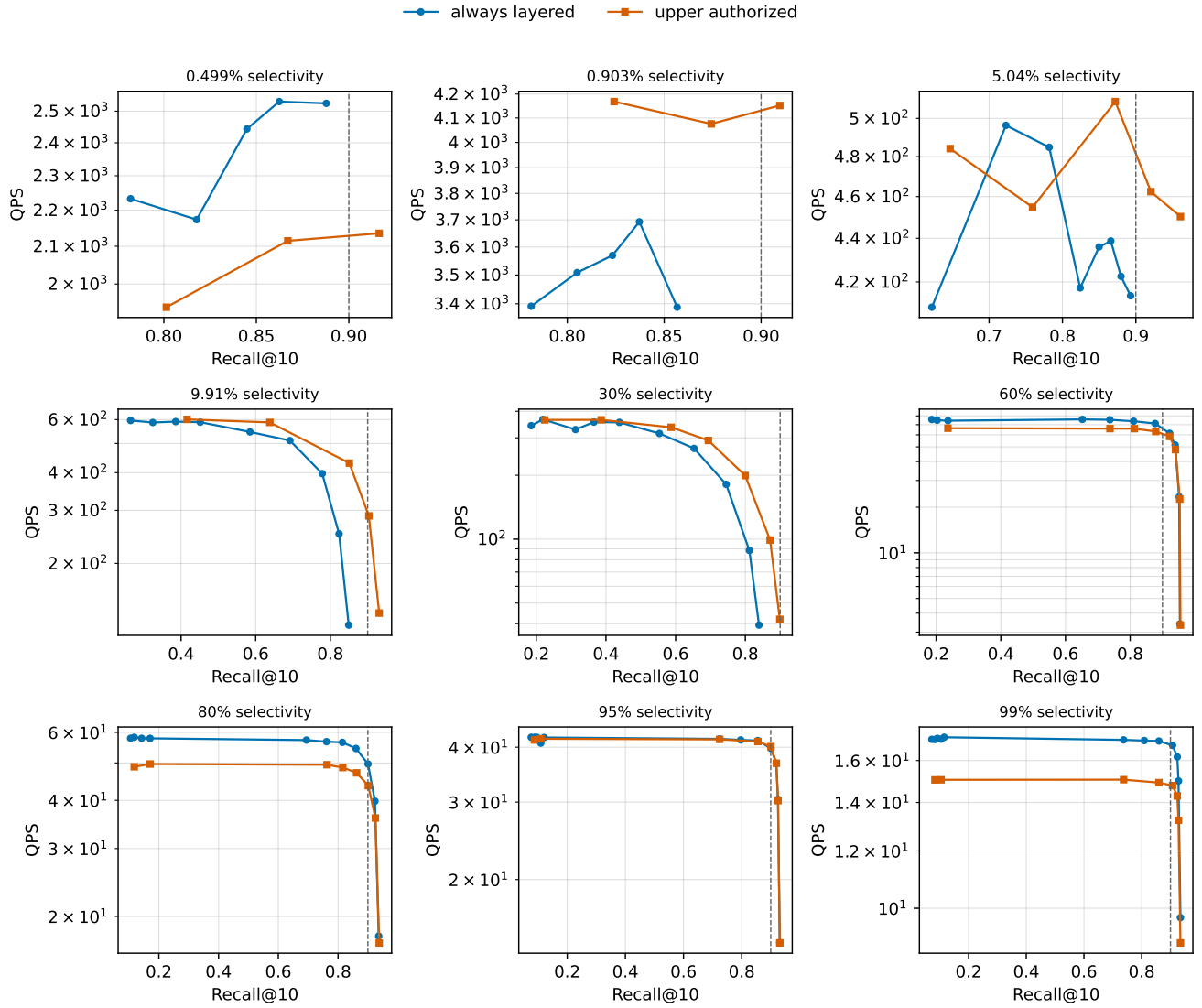


Figure 4: Measured Recall@10-QPS curves for the upper-layer authorization ablation. Markers are executed points; curves are not interpolated.